

Projet DevOps Portfolio

Guide complet : à quoi sert le projet, rôle de chaque fichier,
et instructions pas-à-pas pour le publier sur GitHub

Ce document a été préparé pour t'accompagner, même si tu débutes avec Git et GitHub. Il explique d'abord l'objectif du projet, puis détaille le rôle de chaque fichier, et se termine par un guide étape par étape pour publier le tout sur ton compte GitHub.

1. À quoi sert ce projet ?

Ce projet est un **portfolio DevOps** : un exemple concret et fonctionnel qui démontre, sur un cas simple, l'ensemble des compétences attendues d'un profil SysOps / DevOps junior à confirmé. L'objectif n'est pas la complexité de l'application elle-même (c'est volontairement une petite API), mais de montrer que tu sais mettre en place une **chaîne complète**, de l'infrastructure jusqu'au monitoring, en suivant les bonnes pratiques du métier.

Concrètement, ce projet prouve que tu sais :

- Provisionner une infrastructure de façon déclarative (**Terraform**)
- Configurer un serveur automatiquement, sans action manuelle (**Ansible**)
- Conteneuriser une application (**Docker**)
- Mettre en place une intégration et un déploiement continus (**CI/CD avec GitHub Actions**)
- Intégrer un scan de sécurité automatique dans le pipeline (**Trivy**)
- Superviser une application en production (**Prometheus + Grafana**)

C'est précisément la combinaison de tous ces éléments dans un seul projet, bien documenté, qui fait la différence aux yeux d'un recruteur : ça ne montre pas juste que tu connais un outil, mais que tu comprends comment ils s'articulent entre eux dans un vrai contexte de production.

2. Comment les briques s'articulent

Voici le déroulé logique, du code source jusqu'au monitoring :

1. Tu écris du code -> tu le pousses sur GitHub (git push)
2. GitHub Actions se déclenche automatiquement (CI/CD) :
 - vérifie le code (lint)
 - lance les tests automatiques
 - construit l'image Docker
 - scanne l'image à la recherche de failles de sécurité (Trivy)
 - publie l'image sur un registre (GitHub Container Registry)
3. Terraform crée / gère l'infrastructure (ici : réseau + conteneur Docker)
4. Ansible se connecte à un serveur cible, installe Docker et déploie la stack
5. L'application tourne, Prometheus collecte ses métriques, Grafana les affiche sous forme de graphiques

Un schéma détaillé (format Mermaid, visible directement sur GitHub) est aussi disponible dans le fichier docs/architecture.md.

3. Rôle de chaque fichier et dossier

Racine du projet

README.md

La page d'accueil du dépôt sur GitHub. Présente le projet, la stack technique, comment le lancer, et ce que le projet démontre. C'est le premier fichier que lit un recruteur : il doit donner envie d'explorer

le reste.

LICENSE

Licence MIT : indique que le code est librement réutilisable, avec attribution. Standard sur un portfolio open source.

.gitignore

Liste les fichiers/dossiers que Git ne doit jamais suivre ni envoyer sur GitHub (fichiers temporaires Python, états Terraform contenant des données sensibles, fichiers d'IDE, etc.).

docker-compose.yml

Le chef d'orchestre en local : il démarre en une seule commande l'application, Prometheus et Grafana, déjà connectés entre eux.

Dossier app/ — l'application

app.py

Le code de l'API (Flask). Expose 3 routes : la page d'accueil, un endpoint de vérification de santé (/health), et un endpoint de métriques (/metrics) lu par Prometheus.

requirements.txt

La liste des bibliothèques Python nécessaires à l'application (Flask, client Prometheus, serveur Gunicorn, etc.).

test_app.py

Les tests automatiques de l'application, exécutés à chaque push par la pipeline CI/CD pour vérifier que rien n'est cassé.

Dockerfile

La recette pour construire l'image Docker de l'application : quelle base utiliser, quelles dépendances installer, comment lancer l'app. Écrit selon les bonnes pratiques (utilisateur non-root, build en plusieurs étapes).

Dossier terraform/ — provisionnement de l'infrastructure

main.tf

Décrit l'infrastructure à créer : un réseau et un conteneur Docker pour l'application. Dans un vrai contexte cloud, ce fichier utiliserait un provider AWS/OVHcloud/Scaleway au lieu de Docker, mais la logique reste identique.

variables.tf

Les paramètres modifiables (tag de l'image, port exposé), pour éviter de modifier le code à chaque changement de configuration.

outputs.tf

Les informations affichées après exécution de Terraform (nom du conteneur créé, URL d'accès).

Dossier ansible/ — configuration du serveur

`inventory.ini`

La liste des serveurs sur lesquels Ansible doit intervenir (ici, un exemple à adapter avec l'adresse IP réelle d'un serveur).

`playbook.yml`

La suite d'instructions qu'Ansible exécute automatiquement sur le serveur cible : installer Docker, copier le projet, lancer la stack. Remplace ce qu'un administrateur ferait manuellement, mais de façon reproductible.

Dossier .github/workflows/ — intégration continue

`ci-cd.yml`

Le pipeline GitHub Actions. Déclenché à chaque push : vérifie le code, lance les tests, construit l'image Docker, la scanne avec Trivy, puis la publie. Vérifie aussi que le code Terraform est valide.

Dossier monitoring/ — supervision

`prometheus.yml`

Indique à Prometheus où aller chercher les métriques de l'application (toutes les 5 secondes).

`grafana/provisioning/datasources/datasource.yml`

Connecte automatiquement Grafana à Prometheus au démarrage, sans configuration manuelle dans l'interface.

`grafana/provisioning/dashboards/dashboard.yml`

Indique à Grafana où trouver les tableaux de bord à charger automatiquement.

`grafana/dashboards/devops-portfolio-dashboard.json`

Le tableau de bord prêt à l'emploi : requêtes par seconde, latence, statut de l'application, répartition par code de statut. Visible immédiatement au premier lancement, sans rien configurer.

Dossier docs/ — documentation complémentaire

`architecture.md`

Le schéma d'architecture complet au format Mermaid (s'affiche graphiquement directement sur GitHub), avec l'explication du flux et des choix techniques.

4. Lancer le projet en local (optionnel, pour tester)

Si Docker est installé sur ton ordinateur, tu peux tester le projet avant même de le publier :

```
cd devops-portfolio
docker compose up --build
```

Puis ouvre dans ton navigateur : <http://localhost:5000> (l'application), <http://localhost:9090> (Prometheus), <http://localhost:3000> (Grafana, identifiants admin/admin).

5. Publier le projet sur GitHub — guide pas à pas

Ce guide suppose que tu débutes complètement avec Git. Suis les étapes dans l'ordre.

Étape 1 — Créer un compte GitHub

Si ce n'est pas déjà fait, crée un compte gratuit sur github.com. Choisis un nom d'utilisateur sobre et professionnel : c'est ce que verront les recruteurs.

Étape 2 — Installer Git sur ton ordinateur

Vérifie si Git est déjà installé en ouvrant un terminal et en tapant :

```
git --version
```

Si une version s'affiche, Git est déjà installé. Sinon, télécharge-le depuis git-scm.com/downloads (Windows/Mac) ou installe-le via ton gestionnaire de paquets sous Linux, par exemple :

```
sudo apt install git
```

Étape 3 — Configurer ton identité Git (une seule fois)

Git a besoin de savoir qui tu es pour signer tes contributions :

```
git config --global user.name "Ton Nom"
git config --global user.email "ton.email@exemple.com"
```

Étape 4 — Créer un nouveau dépôt vide sur GitHub

Sur github.com, clique sur le bouton + en haut à droite, puis **New repository**. Donne-lui un nom (par exemple `devops-portfolio`), laisse-le en **Public** (pour que les recruteurs puissent le voir), et surtout **ne coche aucune case** parmi "Add a README", "Add .gitignore" ou "Choose a license" — ces fichiers existent déjà dans ton projet. Clique enfin sur **Create repository**.

Étape 5 — Décompresser le projet reçu

Décompresse l'archive .zip fournie dans le dossier de ton choix, par exemple sur ton Bureau. Tu dois obtenir un dossier nommé `devops-portfolio` contenant tous les fichiers décrits plus haut.

Étape 6 — Initialiser Git et envoyer le projet

Ouvre un terminal, place-toi dans le dossier du projet, puis exécute ces commandes une par une :

```
cd chemin/vers/devops-portfolio

git init
git add .
git commit -m "Premier commit : infra as code, CI/CD, monitoring"
git branch -M main
git remote add origin https://github.com/TON_USERNAME/devops-portfolio.git
git push -u origin main
```

Remplace TON_USERNAME par ton identifiant GitHub, et chemin/vers/devops-portfolio par l'emplacement réel du dossier sur ton ordinateur.

Lors du `git push`, GitHub va te demander de t'authentifier. Depuis 2021, GitHub n'accepte plus le mot de passe classique : il faut soit utiliser un **token d'accès personnel** (Settings -> Developer settings -> Personal access tokens sur GitHub, à utiliser à la place du mot de passe), soit installer l'outil officiel **GitHub CLI** (commande `gh auth login`) qui gère la connexion automatiquement.

Étape 7 — Vérifier que tout fonctionne

Retourne sur la page de ton dépôt GitHub (elle doit maintenant afficher tous tes fichiers), puis clique sur l'onglet **Actions** en haut de la page. Tu devrais voir ta pipeline CI/CD se lancer automatiquement suite au push. Attends quelques minutes : les étapes "lint-and-test" et "terraform-validate" doivent passer au vert.

L'étape "build-scan-push" peut nécessiter d'activer les permissions d'écriture des packages dans les paramètres du dépôt (Settings -> Actions -> General -> Workflow permissions -> "Read and write permissions"). Si tu ne veux pas gérer ça tout de suite, ce n'est pas bloquant : le reste du pipeline fonctionne indépendamment.

Étape 8 — Finaliser la présentation

- Ouvre le fichier README.md sur GitHub et remplace USERNAME dans le lien du badge par ton identifiant GitHub, pour que le badge de statut CI s'affiche correctement.
- Ajoute une description courte au dépôt (bouton engrenage à côté de "About", sur la page du dépôt) et éventuellement des tags/topics comme devops, terraform, ansible, docker, ci-cd.
- Épingles ce dépôt sur la page principale de ton profil GitHub (Customize your pins) pour qu'il soit visible en premier.

6. Pour aller plus loin

Une fois ce projet publié et maîtrisé, quelques évolutions possibles pour continuer à progresser et enrichir ton portfolio :

- Remplacer le provider Terraform Docker par un vrai provider cloud (AWS, OVHcloud, Scaleway) avec un état distant (backend S3 ou équivalent)
- Migrer le déploiement vers Kubernetes (Helm chart)
- Ajouter des tests d'intégration bout en bout dans la pipeline

- Ajouter des alertes Prometheus (Alertmanager) en cas d'anomalie

Document généré pour accompagner le projet devops-portfolio.